Universidad Nacional Autónoma de México

Facultad de Ingeniería

Integrantes:

García Cortés Adolfo de Jesús - 320317264

Lara Hernandez Angel Husiel - 320060829

Lugo Manzano Rodrigo - 320206968

Sistemas Distribuidos

Grupo: 01 - Semestre: 2026-2

Práctica:

RPC y gRPC

Profesora:

M.I. Elba Karen Saenz García

Fecha de Entrega:

4 de Mayo del 2026

Índice

1 Objetivos **1**

2 Investigación Previa **1**

2.1 Que és IDL, XDR, y Protocol Buffers 1

2.2 Tipos de datos a usar en XDR y Protocol Buffers 4

2.3 Significado de los números en archivos .x y .proto 5

3 Desarrollo **7**

4 Actividad 1 . . . . . **7**

4.1 Diseño y Justificación de la Interfaz (IDL) 7

4.2 Generación, Compilación y Ejecución del Proyecto 8

4.3 Desarrollo del Servidor 9

4.4 Desarrollo del Cliente 11

5 Actividad 2 . . . . . **12**

Referencias **12**

1. Objetivos

- Comprender el uso del IDL de Sun-RPC (XDR), el funcionamiento del compilador de interfaces (rpcgen) y el desarrollo de clientes y servidores que hagan uso de las llamadas remotas.
- Comprender el uso de grpc con Python comprendan cómo y definir interfaces de servicios con Protocol Buffers (.proto)

2. Investigación Previa

2.1. Que és IDL, XDR, y Protocol Buffers

IDL (Lenguaje de Definición de Interfaces)

Un Lenguaje de Definición de Interfaces (IDL) es más que un simple formato de texto, es como un contrato entre diferentes sistemas informáticos. En los sistemas distribuidos donde un programa en Python en una computadora necesita pedirle datos a un servidor en C++ en otra computadora, ambos sistemas hablan idiomas diferentes en la memoria. El IDL sirve como un puente universal que describe qué funciones están disponibles para llamarse a través de la red y qué forma exacta tienen los datos que van a viajar.

El verdadero de un IDL esta en su flujo de trabajo automatizado, el cual consta de dos fases principales:

1. **Definición:** El programador escribe un archivo abstracto (por ejemplo, myspec.idl) donde define únicamente las firmas de las funciones (nombre, parámetros y tipo de retorno), sin escribir nada de lógica interna.
2. **Compilación:** Este archivo se pasa por un Çompilador de IDL". El compilador lee el archivo neutro y genera automáticamente código fuente en los lenguajes de programación de destino.

Este código generado se divide en dos piezas clave para la arquitectura de Llamada a Procedimiento Remoto (RPC):

- **El Client Stub (Representante del Cliente):** Es un código que se integra en la aplicación del usuario. Actúa como un proxy local; cuando el cliente llama a una función, el stub intercepta la llamada, empaqueta los datos (proceso de marshaling) y los envía por la red.
- **El Server Skeleton (Esqueleto del Servidor):** Es el código que se integra en el servidor. Su trabajo es escuchar la red, recibir el paquete, desempaquetarlo (proceso de unmarshaling) y pasárselo a la función real que el desarrollador escribió en el servidor.

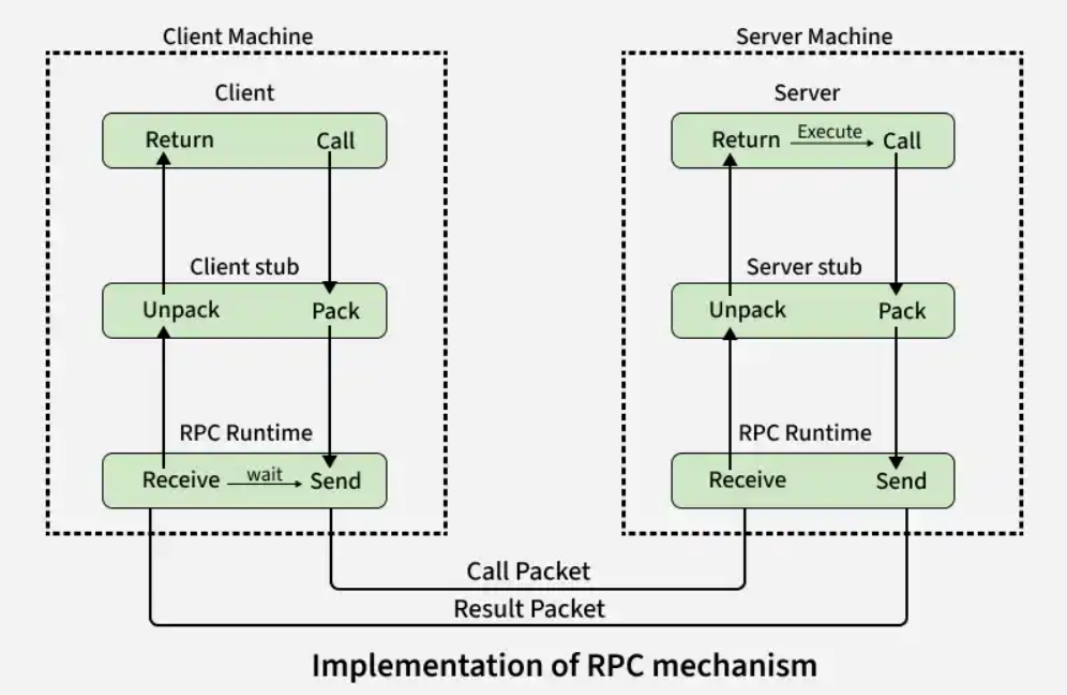

Figura 1: Flujo completo del mecanismo de Llamada a Procedimiento Remoto (RPC), mostrando el empaquetado (marshalling) en el cliente, la transmisión a través del RPC runtime y la ejecución en el servidor mediante el server skeleton.

XDR (Representación Externa de Datos)

La Representación Externa de Datos (XDR) es un estándar de serialización de datos puro y duro. Fue concebido en la década de 1980 por Sun Microsystems y fue la tecnología que hizo posible el famoso sistema de archivos en red NFS (Network File System). Hoy en día está estandarizado bajo el RFC 4506 de la IETF.

El propósito de XDR es resolver el problema de la variedad del hardware, diferentes procesadores almacenan los números en distinto orden (conocido como endianness, como little-endian de Intel vs big-endian de Motorola) y alinean la memoria de formas distintas. Si enviáramos la memoria cruda de una computadora a otra, la segunda interpretaría basura, para solucionarlo, XDR pone unas reglas rígidas sobre cómo convertir cualquier tipo de dato a un formato universal de bytes que pueda ser entendido por cualquier máquina, sin importar su arquitectura:

- **Tipado Implícito:** A diferencia de JSON o XML que son formatos de “tipado explícito"que envían el nombre de la variable junto con el dato, ej: “edad": 25, XDR es de “tipado implícito". En XDR, no viajan metadatos ni etiquetas por el cable, si no que viajan puros ceros y unos. Esto ahorra ancho de banda y tiempo de procesamiento, pero obliga a que tanto el cliente como el servidor tengan exactamente el mismo mapa mental (el archivo .x) para saber cómo leer el flujo de bytes ciego que llega.

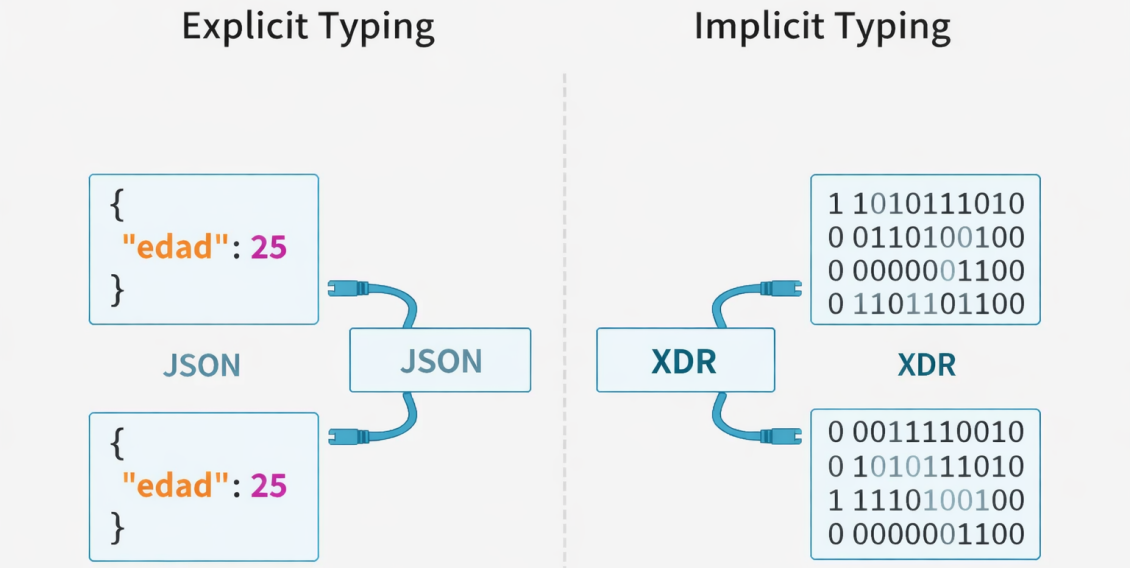

Figura 2: Comparación entre tipado explícito e implícito en serialización de datos.

- **La Regla de Oro de los 4 Bytes:** XDR exige que la unidad básica de todo dato transmitido sea un bloque múltiplo de 4 bytes (32 bits). Los ingenieros eligieron 4 bytes debido a que 2 bytes causaban problemas de alineación en computadoras de la época, y 8 bytes desperdiciaban demasiado espacio.
- **Relleno (Padding):** Si se quiere enviar un tipo de dato pequeño como un simple carácter de 1 byte, XDR no lo envía solo. Forzosamente le inyecta 3 bytes adicionales llenos de ceros (padding) al final para completar el bloque de 4 bytes. Esto garantiza que los datos se alineen perfectamente en la memoria de casi cualquier procesador del mundo, permitiendo que las lecturas sean extremadamente rápidas, aunque sacrifique un poco de espacio en la red.

Protocol Buffers (Protobuf)

Protocol Buffers es el estándar moderno de facto para microservicios. Fue creado por Google para uso interno y luego liberado al público. Protobuf funciona como un IDL (a través de sus archivos .proto) y también es el propio algoritmo de serialización binaria. Es el motor principal detrás de gRPC, el framework de comunicación de alto rendimiento de Google.

Mientras XDR se basa en bloques rígidos de tamaño fijo y posiciones exactas, Protobuf fue diseñado para ser compacto, asimétrico y elástico. Google necesitaba enviar miles de millones de mensajes entre sus servidores usando el menor ancho de banda posible, y al mismo tiempo, poder actualizar los programas añadiendo o quitando campos en los mensajes sin que los servidores viejos colapsaran al no entender los nuevos campos y lo logró mediante dos innovaciones:

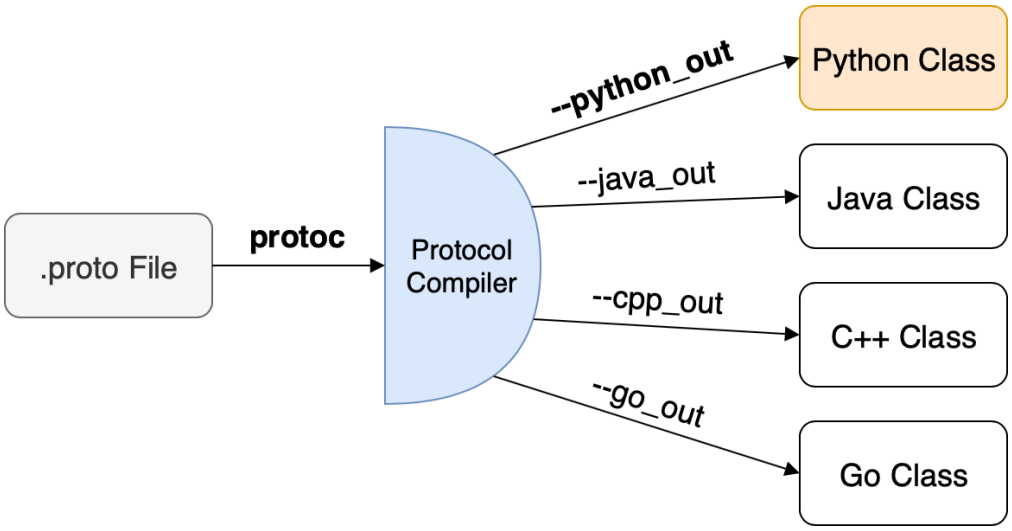

Figura 3: Proceso de compilación en Protocol Buffers, donde un archivo `.proto` es procesado por el compilador `protoc` para generar clases en distintos lenguajes de programación como Python, Java, C++ y Go.

1. **Esquema Tag-Length-Value (TLV):** En lugar de depender de que las variables lleguen en un orden estricto (como XDR), Protobuf anticipa cada variable con una pequeña "Etiqueta" (Tag). Esta etiqueta le dice al receptor: "El siguiente dato es la variable número 1, y es de tipo entero". Si un servidor viejo recibe un mensaje con la etiqueta "variable número 5"que aún no conoce, simplemente lee la longitud del dato, lo salta sin fallar, y sigue leyendo el resto.

2. **Compresión de Enteros Dinámicos (Varints):** En lugar de desperdiciar 4 bytes estáticos para enviar el número 1 como hace XDR obligatoriamente, Protobuf utiliza enteros de longitud variable (Varints). El sistema manipula los bits de tal forma que los números pequeños ocupan solo 1 byte en la red, mientras que números inmensos pueden llegar a ocupar hasta 10 bytes. Para lograrlo, el bit más significativo de cada byte se usa como un semáforo. Si es 1, significa "el número continúa en el siguiente bytez si es 0, significa "el número termina aquí".

2.2. Tipos de datos a usar en XDR y Protocol Buffers

En XDR (RFC 4506)

El lenguaje de XDR se asemeja bastante a C y define los siguientes tipos para su transmisión:

- **Numéricos:** Enteros de 32 bits con y sin signo (`int`, `unsigned int`), enteros largos de 64 bits (`hyper`, `unsigned hyper`), y valores de punto flotante de precisión simple y doble (`float`, `double`).
- **Lógicos:** El tipo booleano (`bool`), el cual también se transmite ocupando un bloque de 32 bits.

- **Estructurados y Compuestos:**
  - Cadenas de caracteres (`string`).
  - Datos binarios sin procesar (`opaque`), ya sea de longitud fija o variable.
  - Colecciones (`arrays` de longitud fija o variable) y estructuras (`struct`).
  - Uniones discriminadas (`union`): A diferencia de las uniones en C, estas asocian directamente un indicador inicial (un número entero o enumeración) llamado "discriminante" que indica qué tipo de estructura se transmitirá a continuación dentro de la unión.
  - Vacío (`void`): Una cantidad de cero bytes, útil para cuando no hay entrada o salida.

En Protocol Buffers (proto3)

El catálogo de tipos está diseñado para maximizar la eficiencia binaria:

- **Estándares de longitud variable:** `int32`, `int64`, `uint32`, `uint64`. El tamaño en bytes que ocupan en red depende de la magnitud del número (varints).
- **Optimizados para signos negativos:** `sint32` y `sint64`. Emplean la codificación "ZigZag" que hace que los números negativos ocupen muchísimo menos espacio que si se enviaran usando el `int32` regular.
- **De tamaño fijo:** `fixed32`, `fixed64`, `sfixed32`, `sfixed64`. Siempre ocupan 4 u 8 bytes exactos. Resultan más eficientes que los varints solo si se transmiten rutinariamente números increíblemente grandes (superiores a 2^{28}).
- **Otros:** Valores de punto flotante (`float`, `double`), booleanos (`bool`), cadenas de texto (obligatoriamente `string` codificado en UTF-8 o ASCII), y arreglos binarios arbitrarios (`bytes`).

2.3. Significado de los números en archivos .x y .proto

A pesar de que ambos utilizan números, el propósito arquitectónico de dichos dígitos es muy diferente en cada ecosistema.

Números en el archivo .x (XDR / ONC RPC)

En el lenguaje RPCL utilizado en XDR, las solicitudes de red hacia procedimientos remotos se enrutan e identifican inequívocamente gracias a una tupla de tres números: (número de programa, número de versión, número de procedimiento).

1. **Número de Programa:** Identifica una agrupación de funciones remotas afines (la aplicación en general). Existen rangos de asignación oficiales regulados por la IANA para evitar colisiones en la red (por ejemplo, el rango hexadecimal `20000000` a `3fffffff` está destinado a ser definido libremente por el usuario).

2. **Número de Versión:** Permite que un mismo programa tenga múltiples versiones conviviendo en el mismo servidor de forma simultánea. Esto ayuda a no romper la compatibilidad con clientes antiguos cuando los desarrolladores añaden nuevos procedimientos.

3. **Número de Procedimiento:** Señala puntualmente la función o método individual que se va a ejecutar en el servidor remoto. Como convención del estándar, el procedimiento `0` suele ser un procedimiento nulo (NULL) que no recibe ni envía datos, usado simplemente para hacer "ping" y probar si el servidor está en línea.

Números en el archivo .proto (Protocol Buffers)

En Protobuf, a cada variable declarada dentro de un mensaje se le debe asignar estrictamente un Número de Campo.

1. **Reemplazo del nombre del campo:** A diferencia de JSON o XML, que repiten el nombre de la variable en el archivo ("nombre", "edad", etc.), Protobuf nunca envía el nombre de la variable en la red. En su lugar, envía únicamente el número de campo para que el receptor sepa en qué ranura encajar el dato, ahorrando enormes cantidades de ancho de banda.

2. **Economía de espacio:** Los números de campo del 1 al 15 ocupan solamente un byte de almacenamiento al serializarse. Por regla de optimización, estos números de oro deben reservarse exclusivamente para las variables que más se usan en una aplicación. Los números del 16 al 2047 pasan a costar dos bytes.

3. **Reglas estrictas de inmutabilidad y exclusión del 0:** Los números de campo deben ser enteros positivos. El número `0` está prohibido. Además, una vez que un número ha sido asignado a un campo y el sistema está en producción, ese número no se puede cambiar ni se debe reutilizar en caso de que el campo sea borrado. Reutilizar un número generaría ambigüedad en el descifrado, corrompiendo datos, provocando fallos en el análisis, y exponiendo potencialmente información confidencial entre sistemas que usan distintas versiones del archivo.

3. Desarrollo

4. Actividad 1

4.1. Diseño y Justificación de la Interfaz (IDL)

El contrato principal de la aplicación distribuida se definió mediante el archivo de especificación `alumnos.x`. Este archivo establece las estructuras de datos y los procedimientos remotos que el servidor expone hacia el programa cliente.

```
struct alumno {
    int id;
    string nombre<>;
    string apellido<>;
    int edad;
    string curso<>;
};

struct busqueda {
    int id;
    string nombre<>;
    string curso<>;
};

union resultado_busqueda switch (int status) {
    case 0:
        alumno al;
    default:
        void;
};

struct alumno_nodo {
    alumno al;
    alumno_nodo *sig;
};

typedef alumno_nodo *lista_alumnos;

program REGISTROALUMNOS {
    version REGISTROALUMNOS_V1 {
        bool registrar_alumno(alumno) = 1;
        resultado_busqueda buscar_alumno(busqueda) = 2;
```

```
            bool actualizar_alumno(alumno) = 3;
            bool eliminar_alumno(int) = 4;
            lista_alumnos listar_alumnos(void) = 5;
        } = 1;
    } = 0x20000001;
```

En la estructura `alumno`, los campos de texto correspondientes al nombre, apellido y curso se definieron como `string<>`. Esta sintaxis indica que son cadenas de longitud variable, lo cual optimiza la transmisión de datos porque XDR solo empaqueta los bytes reales de la cadena más su longitud, en lugar de reservar arreglos estáticos grandes que desperdiciarían ancho de banda con espacios vacíos. La estructura `busqueda` se diseñó con menos campos para permitir consultas flexibles, agrupando únicamente el identificador, el nombre y el curso.

Para el manejo de respuestas complejas, se implementó la estructura condicional `union resultado_busqueda`. Esta unión utiliza una variable entera de estado (status) como discriminante. Cuando la búsqueda es exitosa y el estado es 0, la unión transporta los datos completos del alumno encontrado. Para cualquier otro estado, como una búsqueda fallida, la unión no transporta nada (`void`), ahorrando recursos en la red.

Debido a que Sun-RPC no soporta nativamente la transmisión de listas dinámicas en C como retornos directos, se definió de manera explícita un nodo de lista enlazada llamado `alumno_nodo`. Este nodo contiene la estructura del alumno y un puntero `sig` al siguiente elemento. Al asignarle el alias `lista_alumnos`, el compilador rpcgen entiende que debe generar funciones de serialización recursivas que recorran la lista enlazada nodo por nodo hasta encontrar un puntero nulo, permitiendo transmitir la totalidad de los registros al cliente.

Finalmente, el bloque `program` define la agrupación de todas estas llamadas. Se le asignó el número hexadecimal `0x20000001`. Este valor específico fue seleccionado porque se encuentra dentro del rango reservado por Sun-RPC para aplicaciones definidas por el usuario, evitando conflictos con servicios estándar del sistema operativo. La versión del servicio se fijó en 1, con cada procedimiento remoto enumerado secuencialmente.

4.2. Generación, Compilación y Ejecución del Proyecto

El compilador de protocolos nativo procesa el archivo de interfaz para generar los esqueletos en lenguaje C.

```
rpcgen -a -C alumnos.x
```

La bandera `-a` le indica al compilador que debe generar todos los archivos auxiliares posibles, incluyendo plantillas para el cliente, servidor y un archivo Makefile. La bandera `-C` asegura que el código generado cumpla con el estándar ANSI C moderno.

Debido a que en las distribuciones actuales de Linux las rutinas de RPC fueron extraídas de la biblioteca estándar, es obligatorio utilizar la biblioteca `libtirpc` (Transport Independent RPC). Por esto, el proceso de compilación requiere vincular explícitamente estas dependencias.

```
gcc -c alumnos_client.c -I/usr/include/tirpc
gcc -c alumnos_server.c -I/usr/include/tirpc
gcc alumnos_client.o alumnos_clnt.o alumnos_xdr.o -o cliente -ltirpc
gcc alumnos_server.o alumnos_svc.o alumnos_xdr.o -o servidor -ltirpc
```

Durante la compilación de los códigos fuente, se utiliza la directiva `-I/usr/include/tirpc` para que el compilador encuentre los archivos de cabecera correctos. Durante el enlazado final, la instrucción `-ltirpc` incluye los binarios de la biblioteca para resolver las funciones de red. Para ejecutar el sistema, primero se lanza el servidor en segundo plano o en su propia terminal. Posteriormente, se inicia el cliente pasándole como argumento la dirección IP del equipo donde reside el servidor, utilizando `127.0.0.1` para pruebas locales.

```
./servidor
./cliente 127.0.0.1
```

4.3. Desarrollo del Servidor

El servidor se desarrolló para mantener un control estricto sobre las peticiones y los datos persistentes. Para mejorar la información mostrada al iniciarse el servicio, se modificó el archivo base `alumnos_svc.c` agregando rutinas de red de la API POSIX.

```
if (gethostname(host, sizeof(host)) == 0 &&
    getaddrinfo(host, NULL, &hints, &res) == 0 && res != NULL) {
    (void)getnameinfo(res->ai_addr, res->ai_addrlen, ip, sizeof(ip), NULL,
        0,
        NI_NUMERICHOST);
    freeaddrinfo(res);
}

printf("[SISTEMA] Servidor RPC iniciado correctamente. Escuchando
   peticiones en IP %s (Puerto asignado por rpcbind)\n", ip);
fflush(stdout);
```

El proceso obtiene primero el nombre del equipo anfitrión a través de la función `gethostname`. Con este nombre, se invoca a `getaddrinfo` pasándole una estructura `hints` previamente configurada para buscar direcciones IPv4 (`AF_INET`) aptas para recibir conexiones entrantes (`SOCK_STREAM` y `AI_PASSIVE`). Si la resolución fue exitosa, se utiliza `getnameinfo` sobre la estructura resultante para traducir esa configuración a

un nombre numérico válido, es decir, la dirección IP textual, la cual se almacena en la variable `ip`. Finalmente, se imprimen estos datos en la consola para confirmar al administrador que el servicio está activo y disponible en la red local, utilizando `fflush` para vaciar de inmediato el búfer de salida.

Para llevar un registro estadístico de los clientes, se implementó una función transversal `log_peticion` en `alumnos_server.c`.

```c
static void log_peticion(struct svc_req *rqstp, const char *operacion)
{
    char ip[NI_MAXHOST] = "desconocida";
    char puerto[NI_MAXSERV] = "0";

    if (rqstp != NULL && rqstp->rq_xprt != NULL) {
        struct netbuf *caller = &rqstp->rq_xprt->xp_rtaddr;
        if (caller->buf != NULL && caller->len > 0) {
            struct sockaddr_storage ss;
            socklen_t salen;
            size_t copiar;

            memset(&ss, 0, sizeof(ss));
            copiar = caller->len < sizeof(ss) ? caller->len : sizeof(ss);
            memcpy(&ss, caller->buf, copiar);
            salen = (socklen_t)copiar;

            if (getnameinfo((struct sockaddr *)&ss, salen, ip, sizeof(ip),
                puerto,
                sizeof(puerto), NI_NUMERICHOST | NI_NUMERICSERV) != 0) {
                strcpy(ip, "desconocida");
                strcpy(puerto, "0");
            }
        }
    }

    printf("[RECEPCION] Peticion de %s:%s para la operacion %s\n", ip,
        puerto,
        operacion != NULL ? operacion : "desconocida");
    fflush(stdout);
}
```

Esta función extrae los metadatos de la estructura de la solicitud entrante `rqstp`. Valida que existan los punteros asociados a la capa de transporte y copia los bytes sin procesar contenidos en `xp_rtaddr` hacia una estructura `sockaddr_storage` de la máquina local. Con la dirección ya tipada localmente, la función

de resolución extrae el host numérico y el puerto de origen exactos de quien realizó la llamada remota, permitiendo dejar un registro descriptivo en la salida estándar cada vez que un servicio es invocado.

En cuanto al almacenamiento, las funciones de persistencia abren iterativamente el archivo de texto plano `alumnos_db.txt`. Leen cada línea entera usando `fgets` y delegan a una subrutina la tokenización del texto mediante la función `strtok_r`, la cual divide cada cadena detectando el carácter separador `|`. En los casos de eliminación o actualización parcial, el archivo se lee secuencialmente al mismo tiempo que los datos válidos u omitidos se reescriben de inmediato sobre un archivo temporal. Una vez finalizada la revisión total de las entradas, el proceso invoca las llamadas al sistema necesarias para desvincular el archivo viejo y renombrar el temporal como oficial, asegurando la atomicidad de los cambios.

4.4. Desarrollo del Cliente

En el código principal del cliente (`alumnos_client.c`), la ejecución arranca verificando la disponibilidad de conexión lógica mediante el método `clnt_create`, al cual se le proporciona la IP enviada por la consola, el nombre del programa, la versión y el protocolo de red subyacente TCP. Una vez establecido el canal, inicia el ciclo eterno que dibuja el sistema de captura de entradas guiadas.

Para prevenir búsquedas, actualizaciones o eliminaciones que lógicamente no tienen sentido realizar sin información existente en el servidor, se forzó la llamada estricta a `listar_todos` en los casos respectivos del menú principal.

```
    case 3:
        if (listar_todos(clnt) <= 0) {
            break;
        }
        printf("\nDeja un campo vacio y presiona Enter para conservar su valor
            actual.\n");
        al.id = leer_entero("ID del alumno a actualizar: ");
        leer_linea("Nuevo nombre: ", nombre, sizeof(nombre));
        leer_linea("Nuevo apellido: ", apellido, sizeof(apellido));
        al.edad = leer_entero_opcional("Nueva edad (Enter para conservar): ",
            -1);
```

La función de listado solicita incondicionalmente todos los registros a través de XDR y retorna un código numérico. Al validar el resultado, si el código determina que no hay alumnos (por listar un apuntador nulo), el condicional invoca la instrucción `break` forzando el abandono inmediato de la opción actual. El resultado retorna al usuario nuevamente el menú, bloqueando el avance y previniendo que la aplicación exija repetitivamente datos si la base se encuentra en blanco.

Si la lista de valores sí contiene información, se prosigue con un bloque interactivo inteligente. La lec-

tura de argumentos en consola se hace con funciones delegadas diseñadas a medida como `leer_linea` o `leer_entero_opcional`. Si el usuario entra directamente un retorno de carro (presionando `Enter` de inmediato) al solicitar el parámetro o el bloque de texto en cuestión, la cadena provista se registra vacía o bien el analizador entero lo intercambia por un número preconfigurado que actúa como bandera centinela, comúnmente posicionado en `-1`. Este conjunto es transmitido de idéntica manera al servidor en el paquete de la solicitud. En el ciclo homólogo de actualización en el servidor, este repasa minuciosamente cada variable integrante. Si capta el marcador `-1` o la ausencia de caracteres exactos en los apuntadores serializados de cadena, salta asíncronamente su actualización particular y opta estrictamente por retener en el registro definitivo la información precedente extraída del flujo local de persistencia.

5. Actividad 2

Referencias

Amazon Web Services. (s.f.). *Manage Amazon EC2 instance clock accuracy using Amazon Time Sync Service*. https://aws.amazon.com/blogs/mt/manage-amazon-ec2-instance-clock-accuracy-using-amazon-time-sync-service-and-amazon-cloudwatch-part-1/

Cisco Systems. (2026, 17 de febrero). *NTP Configuration Guide - Network Time Protocol [Cisco IOS XE 17]*. https://www.cisco.com/c/en/us/td/docs/switches/lan/c9000/time-sync/ntp-configuration-guide/m-network-time-protocol.html

Schmuck, F., & Cristian, F. (1990). *Probabilistic approach to distributed clock synchronization*. https://research.ibm.com/publications/probabilistic-approach-to-distributed-clock-synchronization